



MCP Architecture : All you need

11 min read · Oct 8, 2025



Sweety Tripathi

Follow



Listen



Share

The Model Context Protocol (MCP) is a modern, standardized way to let large language models (LLMs) seamlessly connect with external tools and real-world data sources gives new levels of capability, flexibility, and maintainability for AI-centric applications.

MCP Architecture

Host

Client

Server

External
Resources/Tools ClaudeMCP
Client

MCP PROTOCOL

MCP
Client

MCP PROTOCOL

MCP
Client

MCP PROTOCOL

slack
from Salesforce

Read this before you jump up in this article.

Why Learn MCP? The Future of AI Tool Integration

MCP is a term you've probably seen around on social media, with plenty of debates but little clarity on what it...

medium.com

Core Components of MCP Architecture

The Model Context Protocol (MCP) is built on a **client-server architecture** involving three main parts.

1. First, there's the **MCP Host**. This is the AI application you interact with, like Visual Studio Code, Claude Desktop, or any AI tool. The host is responsible for managing the overall experience and coordinating communication.
2. Next comes the **MCP Client**. Each MCP client is created by the host to connect to a specific MCP server. They have a one-to-one relationship one client talks to one server. The client's job is to maintain that connection and get useful context or data from the server for the host to use.
3. Finally, there's the **MCP Server**. This is the program or service that provides the context or data that AI applications need. It could be something like a server that handles your local files or one like the Sentry MCP server that offers monitoring data.

For example, when Visual Studio Code connects to the Sentry MCP server, it creates a dedicated MCP client to manage that connection. If it also connects to another server like a local filesystem server, it spins up another dedicated MCP client for that server. So, each server has its own client, keeping connections clear and organized.

In short, the MCP architecture works by having AI applications (hosts) create individual clients that maintain separate connections to different servers, enabling smooth and modular interactions.

Lets Talk about every component in detail

Host

The Host is the main user-facing application a platform where the user's interaction starts.

Examples of MCP Hosts:

- **Claude Desktop** — Anthropic's desktop application
- **Visual Studio Code** — With MCP extensions
- **Cursor** — AI-enhanced IDE
- **Custom AI applications** — Built by developers



- This could be an AI-powered chat interface, a productivity tool (like Vs Code or Claude Desktop), or an AI-enhanced Integrated Development Environment (IDE) such as Cursor.
- The host manages user inputs, displays responses, and generally steers the overall flow of information between the user, LLM, and any external data sources or tools.

Client

The Client acts as a translator and communication manager.

- It resides inside the host, encoding user intents into structured protocol messages and then securely transmitting them to the MCP Server.
- The client makes sure every message follows the MCP specification, handles connections, and manages transport whether the application is running entirely locally or communicating with server resources remotely.
- Typically, every client connects to one server, maintaining a clear, auditable flow.

Key Characteristics:

- **One-to-one relationship:** Each client connects to exactly one server
- **Protocol enforcement:** Ensures all messages follow MCP specification
- **Connection management:** Handles transport, security, and error recovery
- **Message translation:** Converts host intents into structured MCP requests

```
import asyncio
import json
from typing import Dict, Any, List
from pathlib import Path
from fastmcp import MCPClient

class FileManagerClient:
    """
    Client for interacting with File Manager MCP Server

    Provides high-level interface for AI assistants to perform
    file operations through the MCP protocol.
    """

    def __init__(self, server_command: List[str]):
        self.client = MCPClient(server_command)
        self.connected = False

    async def connect(self):
        """Connect to the MCP server"""
        try:
            await self.client.connect()
            self.connected = True
            print("✅ Connected to File Manager MCP Server")

            # List available tools
            tools = await self.client.list_tools()
            print(f"📋 Available tools: {[tool['name'] for tool in tools.get('tools', [])]}")

        except Exception as e:
            print(f"❌ Failed to connect: {e}")
            self.connected = False

    async def read_file(self, file_path: str) -> Dict[str, Any]:
        """Read a file through MCP"""
        if not self.connected:
            return {"error": "Not connected to server"}
```

```

    try:
        result = await self.client.call_tool("read_file", {"file_path": file_path})
        return result
    except Exception as e:
        return {"error": f"Failed to read file: {e}"}

    async def write_file(self, file_path: str, content: str) -> Dict[str, Any]:
        """Write a file through MCP"""
        if not self.connected:
            return {"error": "Not connected to server"}

        try:
            result = await self.client.call_tool("write_file", {
                "file_path": file_path,
                "content": content
            })
            return result
        except Exception as e:
            return {"error": f"Failed to write file: {e}"}

```

Server

The Server is an integration and execution layer the digital “back-of-house” for AI applications.

- It connects with databases, APIs, file systems, knowledge bases, and specialized tools to fetch, compute, or transform information.
- The server processes requests from clients, interacts with external resources as needed, and responds with structured, machine-readable context making it easy for the LLM to use the retrieved information directly in its reasoning.

Key Characteristics:

- Connecting with databases, APIs, file systems, and specialized tools
- Processing requests from clients and fetching required data
- Providing structured, machine-readable responses
- Exposing capabilities through tools, resources, and prompts

```

import asyncio
from typing import Dict, Any
from fastmcp import FastMCP

```

```

class CalculatorMCPServer:
    """
    Simple calculator MCP server

    Provides basic arithmetic operations as MCP tools
    that AI assistants can use for calculations.
    """

    def __init__(self):
        self.mcp = FastMCP("Calculator Server")
        self._setup_tools()

    def _setup_tools(self):
        """Register calculator tools"""

        @self.mcp.tool()
        async def add(a: float, b: float) -> Dict[str, Any]:
            """
            Add two numbers

            Args:
                a: First number
                b: Second number

            Returns:
                Sum of a and b
            """
            try:
                result = a + b
                return {
                    "operation": "addition",
                    "inputs": {"a": a, "b": b},
                    "result": result,
                    "formatted": f"{a} + {b} = {result}"
                }
            except Exception as e:
                return {"error": f"Addition failed: {str(e)}"}

        @self.mcp.tool()
        async def solve_quadratic(a: float, b: float, c: float) -> Dict[str, Any]:
            """
            Solve quadratic equation  $ax^2 + bx + c = 0$ 

            Args:
                a: Coefficient of  $x^2$ 
                b: Coefficient of  $x$ 
                c: Constant term

            Returns:
                Solutions to the quadratic equation
            """
            try:

```

```

import math

if a == 0:
    return {"error": "Not a quadratic equation (a cannot be 0)"}

discriminant = b**2 - 4*a*c

if discriminant < 0:
    return {
        "operation": "quadratic_equation",
        "equation": f"{a}x2 + {b}x + {c} = 0",
        "discriminant": discriminant,
        "solutions": "No real solutions (complex roots)",
        "note": "Discriminant is negative"
    }
elif discriminant == 0:
    solution = -b / (2*a)
    return {
        "operation": "quadratic_equation",
        "equation": f"{a}x2 + {b}x + {c} = 0",
        "discriminant": discriminant,
        "solutions": [solution],
        "formatted": f"x = {solution} (repeated root)"
    }
else:
    sqrt_discriminant = math.sqrt(discriminant)
    solution1 = (-b + sqrt_discriminant) / (2*a)
    solution2 = (-b - sqrt_discriminant) / (2*a)
    return {
        "operation": "quadratic_equation",
        "equation": f"{a}x2 + {b}x + {c} = 0",
        "discriminant": discriminant,
        "solutions": [solution1, solution2],
        "formatted": f"x = {solution1} or x = {solution2}"
    }

except Exception as e:
    return {"error": f"Quadratic equation solving failed: {str(e)}"}

async def run(self):
    """Run the calculator MCP server"""
    print("🧮 Calculator MCP Server starting...")
    print("Available tools: add, multiply, calculate_percentage, solve_quadratic")
    await self.mcp.run(transport="stdio")

```

Marketplaces

Here are the ones to watch:

Get Sweety Tripathi's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

[mcpmarket.com](#) — A plug-and-play directory of MCP servers for tools like GitHub, Figma, Notion, Databricks, and more.

[mcp.so](#) — A growing open repo of community-built MCP servers. Discover one. Fork it. Build your own.

[Cline's MCP Marketplace](#) — A GitHub-powered hub for open-source MCP connectors anyone can use.

GitHub - modelcontextprotocol/servers: Model Context Protocol Servers

Model Context Protocol Servers. Contribute to modelcontextprotocol/servers development by creating an accoun...

github.com

How MCP Components Communicate

Communication in MCP is built on JSON-RPC 2.0, which provides a clear, predictable structure for requests, responses, and notifications.

MCP supports two main transport mechanisms:

- **STDIO** : for local integrations, where the server runs on the same device as the client.
- **HTTP + SSE (Server-Sent Events)** : for remote deployments, allowing hosts and servers to be decoupled and run anywhere cloud, local, or hybrid setups.

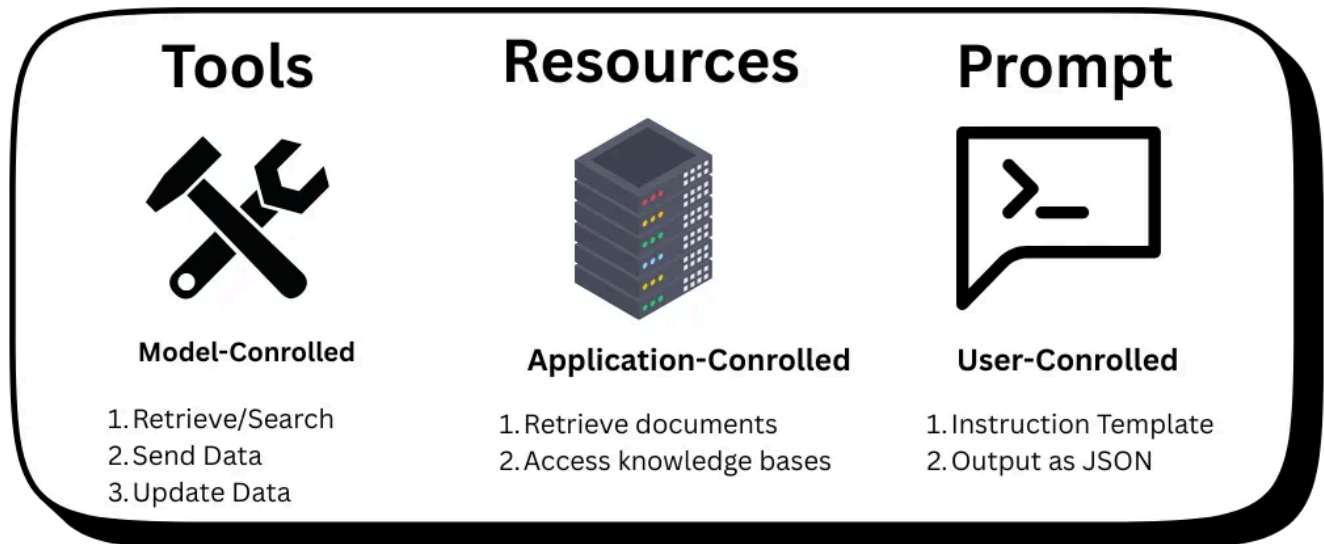
The Lifecycle of an MCP Request

A typical workflow looks like this:

- User submits a query via the host application.
- The client in the host encodes this request, ensuring it follows the MCP format.
- The server receives the request, fetches the necessary data or performs actions using its integrated tools, resources, or prompts (designed for retrieval-augmented or agentic systems).
- The server sends the response structured and LLM-friendly back to the client.
- The host presents this result to the user, closing the loop.

MCP Server Primitives: Tools, Resources, and Prompts

Central to MCP's power is the server's ability to offer capability in three ways:



1. Tools: Executable Functions and Actions

Tools are executable functions that enable agent-based workflows and side effects.

Think of tools as POST endpoints they perform actions, make changes, or execute operations.

```
from fastmcp import FastMCP
from pydantic import BaseModel, Field
import asyncio
import os
from pathlib import Path
from typing import List, Dict, Any, Optional
```

```
class FileManagerServer:
    """
```

Secure file management server for AI assistants

Features:

- Sandboxed file operations
- Permission checking
- Safe path validation
- File type restrictions

"""

```
def __init__(self, allowed_directories: List[str], max_file_size: int = 10_
    self.allowed_directories = [Path(d).resolve() for d in allowed_director
    self.max_file_size = max_file_size
    self.mcp = FastMCP("File Manager Server")
    self._setup_tools()
```

```
def _is_path_allowed(self, file_path: str) -> bool:
    """Check if file path is within allowed directories"""
    try:
        resolved_path = Path(file_path).resolve()
        return any(
            str(resolved_path).startswith(str(allowed_dir))
            for allowed_dir in self.allowed_directories
        )
    except Exception:
        return False
```

```
def _setup_tools(self):
    """Register MCP tools"""

    @self.mcp.tool()
    async def read_file(file_path: str) -> Dict[str, Any]:
        """
        Read file contents safely

        Args:
            file_path: Path to the file to read

        Returns:
            Dictionary with file content and metadata
        """
        if not self._is_path_allowed(file_path):
            return {"error": "Access denied: Path not in allowed directorie

        try:
            path = Path(file_path)
            if not path.exists():
                return {"error": f"File not found: {file_path}"}

            if path.stat().st_size > self.max_file_size:
                return {"error": f"File too large (max {self.max_file_size}

            # Handle different file types
            import mimetypes
```

```

mime_type, _ = mimetypes.guess_type(file_path)

if mime_type and mime_type.startswith('text/'):
    with open(path, 'r', encoding='utf-8') as f:
        content = f.read()
    return {
        "content": content,
        "type": "text",
        "mime_type": mime_type,
        "size": len(content),
        "path": file_path
    }
else:
    # For binary files, return base64
    import base64
    with open(path, 'rb') as f:
        content = base64.b64encode(f.read()).decode('utf-8')
    return {
        "content": content,
        "type": "binary",
        "encoding": "base64",
        "mime_type": mime_type,
        "size": path.stat().st_size,
        "path": file_path
    }

except Exception as e:
    return {"error": f"Failed to read file: {str(e)}"}

@self.mcp.tool()
async def write_file(file_path: str, content: str, mode: str = "w") ->
    """
    Write content to file safely

    Args:
        file_path: Path where to write the file
        content: Content to write
        mode: Write mode ('w' for text, 'wb' for binary)

    Returns:
        Success status and file information
    """
    if not self._is_path_allowed(file_path):
        return {"error": "Access denied: Path not in allowed directorie

    try:
        path = Path(file_path)

        # Create parent directories if they don't exist
        path.parent.mkdir(parents=True, exist_ok=True)

        # Check content size
        if len(content.encode('utf-8')) > self.max_file_size:

```

```

        return {"error": f"Content too large (max {self.max_file_si

# Write file
if mode == "wb":
    # Handle base64 binary content
    import base64
    binary_content = base64.b64decode(content)
    with open(path, 'wb') as f:
        f.write(binary_content)
else:
    with open(path, 'w', encoding='utf-8') as f:
        f.write(content)

return {
    "success": True,
    "message": f"File written successfully: {file_path}",
    "size": path.stat().st_size
}

except Exception as e:
    return {"error": f"Failed to write file: {str(e)}"}

```

2. Resources: Read-Only Data Access

Resources provide access to files, documents, knowledge repositories, and live data for retrieval-augmented generation (RAG).

Think of resources as GET endpoints they provide information without side effects.

```

import json
import mimetypes
from datetime import datetime

```

```

@self.mcp.tool()
async def list_directory(directory_path: str, include_hidden: bool = False) ->
    """
    List directory contents

    Args:
        directory_path: Path to directory to list
        include_hidden: Whether to include hidden files

    Returns:
        List of files and directories with metadata
    """

```

```

if not self._is_path_allowed(directory_path):
    return {"error": "Access denied: Path not in allowed directories"}

try:
    path = Path(directory_path)
    if not path.exists():
        return {"error": f"Directory not found: {directory_path}"}

    if not path.is_dir():
        return {"error": f"Path is not a directory: {directory_path}"}

    files = []
    for item in path.iterdir():
        if not include_hidden and item.name.startswith('.'):
            continue

        stat = item.stat()
        mime_type, _ = mimetypes.guess_type(str(item))

        file_info = {
            "name": item.name,
            "path": str(item),
            "size": stat.st_size,
            "modified": datetime.fromtimestamp(stat.st_mtime).isoformat(),
            "is_directory": item.is_dir(),
            "mime_type": mime_type
        }
        files.append(file_info)

    return {
        "directory": directory_path,
        "files": files,
        "count": len(files)
    }

except Exception as e:
    return {"error": f"Failed to list directory: {str(e)}"}

@self.mcp.tool()
async def search_files(
    directory_path: str,
    pattern: str,
    file_type: Optional[str] = None,
    max_results: int = 100
) -> Dict[str, Any]:
    """
    Search for files matching pattern

    Args:
        directory_path: Directory to search in
        pattern: Search pattern (supports wildcards)
        file_type: Filter by file extension (e.g., '.py', '.txt')
        max_results: Maximum number of results
    """

```

```

Returns:
    Search results with file information
"""
if not self._is_path_allowed(directory_path):
    return {"error": "Access denied: Path not in allowed directories"}

try:
    import fnmatch

    path = Path(directory_path)
    if not path.exists() or not path.is_dir():
        return {"error": f"Invalid directory: {directory_path}"}

    matching_files = []

    for item in path.rglob('*'):
        if len(matching_files) >= max_results:
            break

        if item.is_file():
            # Check pattern match
            if fnmatch.fnmatch(item.name.lower(), pattern.lower()):
                # Check file type filter
                if file_type and not item.name.lower().endswith(file_type.lower()):
                    continue

                stat = item.stat()
                file_info = {
                    "name": item.name,
                    "path": str(item),
                    "size": stat.st_size,
                    "modified": datetime.fromtimestamp(stat.st_mtime).isoformat()
                }
                matching_files.append(file_info)

    return {
        "search_query": pattern,
        "directory": directory_path,
        "file_type_filter": file_type,
        "results": matching_files,
        "count": len(matching_files),
        "truncated": len(matching_files) >= max_results
    }

except Exception as e:
    return {"error": f"Search failed: {str(e)}"}

```

3. Prompts: Reusable Interaction Templates

Prompts are predefined instruction templates that guide model outputs and reduce the need for user prompt engineering.

```
from mcp.types import Prompt, PromptArgument, PromptMessage
```

```
@app.list_prompts()
async def list_prompts() -> list[Prompt]:
    """Define reusable prompt templates"""
    return [
        Prompt(
            name="code_review",
            description="Comprehensive code review template",
            arguments=[
                PromptArgument(
                    name="code",
                    description="The code to review",
                    required=True
                ),
                PromptArgument(
                    name="language",
                    description="Programming language",
                    required=False
                ),
                PromptArgument(
                    name="focus_areas",
                    description="Specific areas to focus on (security, performance, etc)",
                    required=False
                )
            ]
        ),
        Prompt(
            name="data_analysis",
            description="Data analysis and insights template",
            arguments=[
                PromptArgument(
                    name="dataset",
                    description="The dataset to analyze",
                    required=True
                ),
                PromptArgument(
                    name="analysis_type",
                    description="Type of analysis (descriptive, predictive, etc)",
                    required=False
                )
            ]
        ),
        Prompt(
            name="general_inquiry",
            description="General inquiry and information template",
            arguments=[
                PromptArgument(
                    name="topic",
                    description="The topic of inquiry",
                    required=True
                ),
                PromptArgument(
                    name="context",
                    description="Additional context or background information",
                    required=False
                )
            ]
        )
    ]
```



```

name="technical_documentation",
description="Generate technical documentation",
arguments=[
    PromptArgument(
        name="topic",
        description="The topic to document",
        required=True
    ),
    PromptArgument(
        name="audience",
        description="Target audience (developers, users, etc.)",
        required=False
    ),
    PromptArgument(
        name="format",
        description="Documentation format (API, tutorial, reference)",
        required=False
    )
]
)
]
)
]
@app.get_prompt()
async def get_prompt(name: str, arguments: dict) -> PromptMessage:
    """Generate prompt content based on template and arguments"""
    if name == "code_review":
        code = arguments["code"]
        language = arguments.get("language", "unknown")
        focus_areas = arguments.get("focus_areas", "general best practices")
        prompt_content = f"""
        Please conduct a thorough code review of the following {language} code.
        Focus Areas: {focus_areas}
        Code to Review:
        ```{language}
 {code}
        ```
        """

```

Why MCP Matters

- You can run everything : host, client, and server on a single device for privacy and speed, or split the server off to a cloud environment for scalability and high availability.
- The components can be mixed, matched, and upgraded independently, giving developers enormous control over integration depth, update cycles, and security boundaries.
- Scalability: Built once, used everywhere any compliant host or server can interoperate.

- **Maintainability:** Standardized interfaces make upgrades, troubleshooting, and onboarding much easier.
- **Vendor Neutrality & Openness:** Avoids lock-in; mix and match components as needed.
- **Speed:** Rapid development thanks to mature building blocks, removing the need for bespoke integrations for each new tool or dataset.

Conclusion

You can run your code absolutely without MCP but *why reinvent the wheel*. The Model Context Protocol isn't just a technical framework it's a new foundation for context-rich AI tool.

Thank you for reading! 😊 I hope that you found this article both informative and enjoyable to read.

For more information like this follow me on [LinkedIn](#)

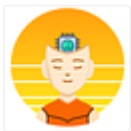
Mcp Server

AI

ChatGPT

Llm Applications

LLM



Follow

Published in AI monks.io

6.4K followers · Last published 3 hours ago

Almonks (<https://medium.com/aimonks>) is an AI-Educational Publication.



Follow

Written by Sweety Tripathi

303 followers · 23 following

Data Scientist | Generative AI Engineer | Writing about how AI learns and reasons. From coding models to decoding intelligence.

